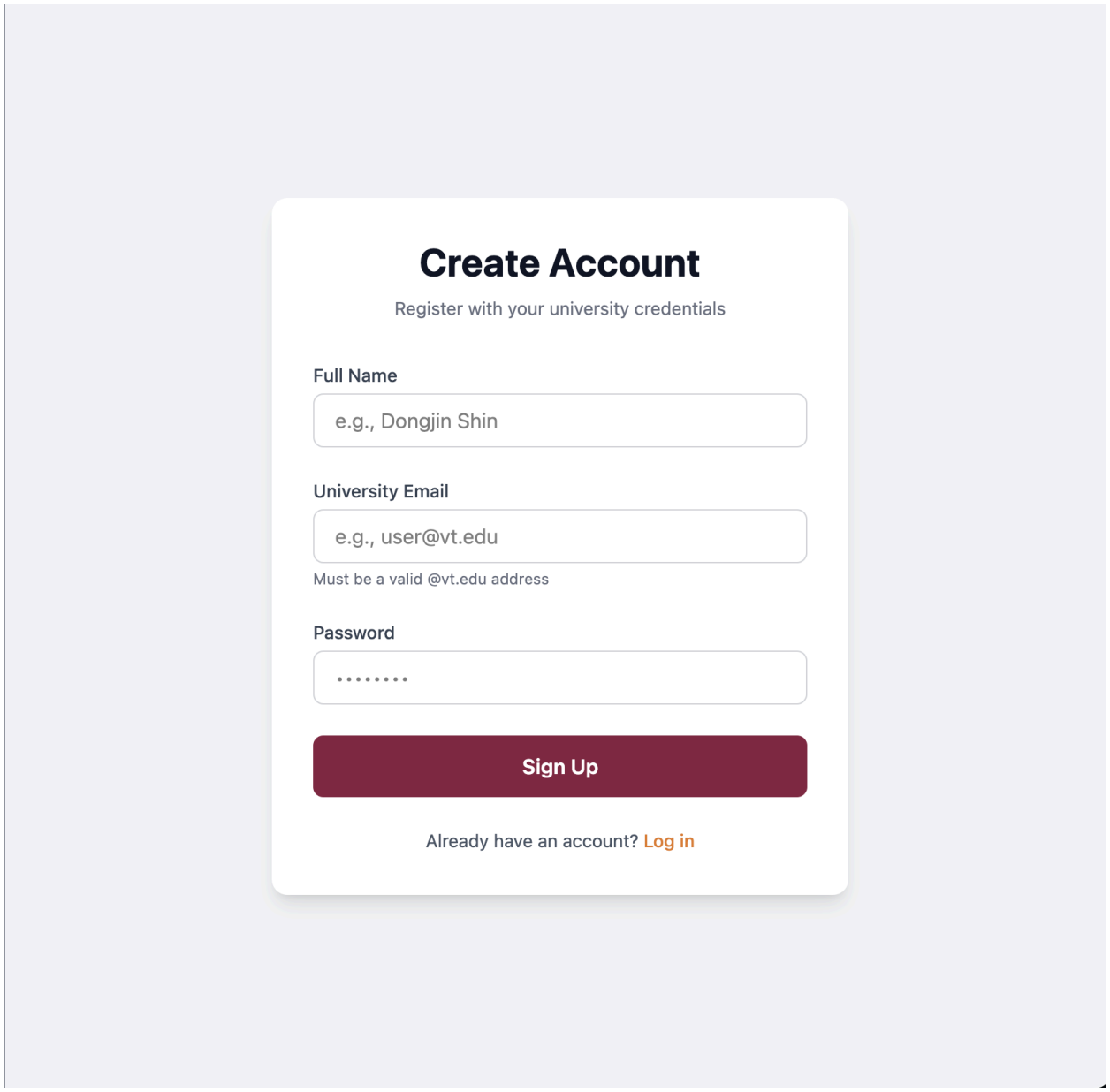
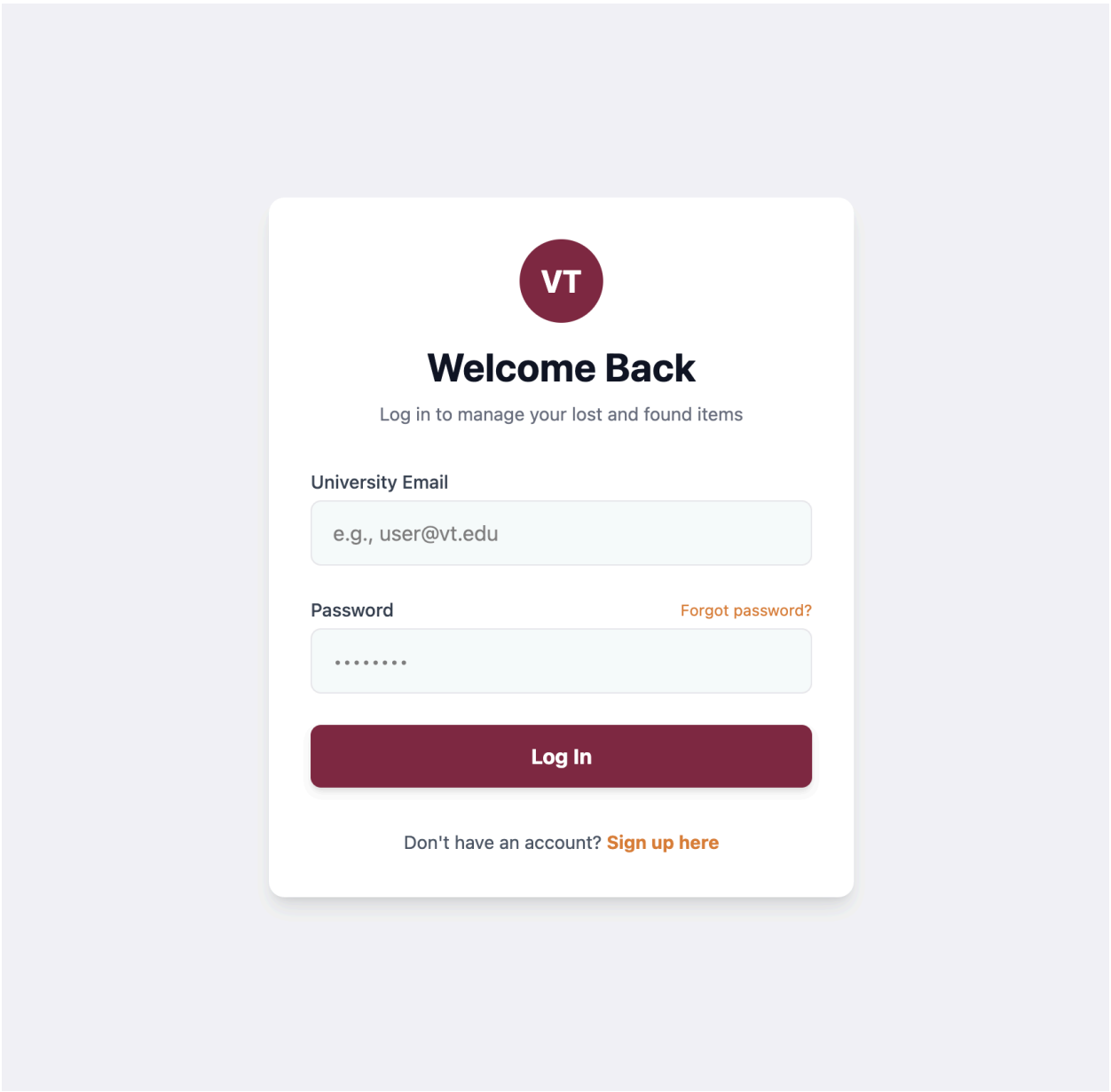
1. **UI Design or Mock UIs:**




VT
Welcome Back
Log in to manage your lost and found items
University Email
e.g., user@vt.edu
Password
Forgot password?
••••••••
Log In
Don't have an account? Sign up here

Report a Lost Item

Provide details so the community can help find it.

Item Title

e.g., Black Hydroflask

Category

Electronics

Date Lost

mm/dd/yyyy

Location Last Seen

e.g., Torgersen Hall, 3rd Floor

Description & Identifying Marks

Has a specific sticker, scratch on the left side...

Upload Photos (Optional, 0-5 max)

Submit Report

Find an Item

Search 'AirPods', 'Keys'...

Search

Status: Available | Type: Found | Category: Electronics +

Showing 2 results

AirPods Pro (Gen 2)

Found

Found near the library entrance. White case with a minor scratch.

Newman Library • 2 hours ago

Blue North Face Backpack

Lost

Left it in a classroom. Contains notebooks and a charger.

McBryde Hall • Yesterday

← Back to My Found Items

Claim Request Review

For: Blue North Face Backpack

Pending Review

CLAIMANT DETAILS

JS

John Smith

Requested 2 hours ago

Message User

SUBMITTED PROOF OF OWNERSHIP

"There is a small tear on the right shoulder strap, and the front pocket contains a VT math notebook with my name on it."

Decline Claim

Accept & Verify Proof

My Dashboard

+ New Post

My Lost Items | My Found Items

iPhone 13 with Red Case Pending Claim (1)

Reported Lost on Oct 12 • Squires Student Center

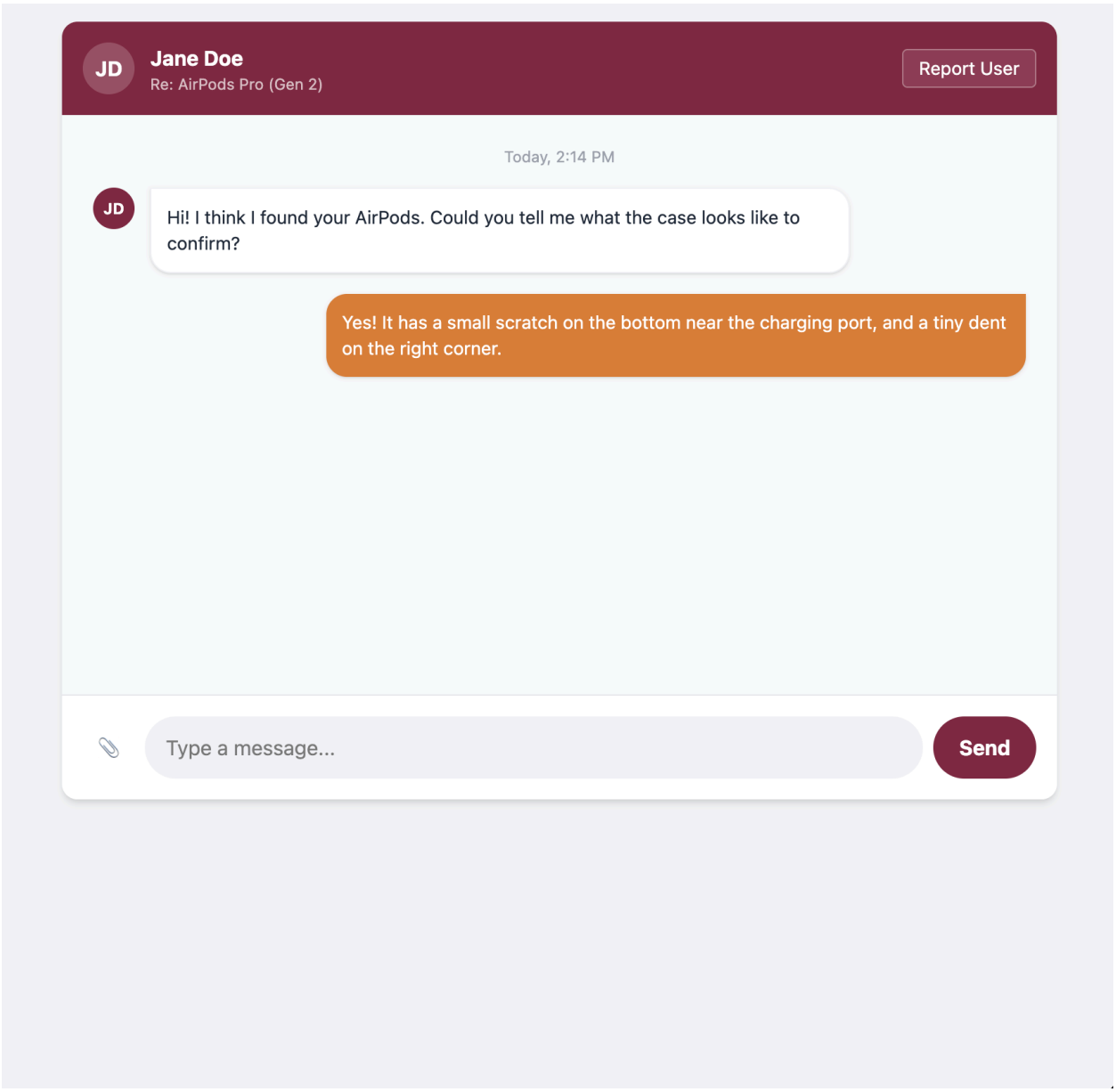
Review Claim

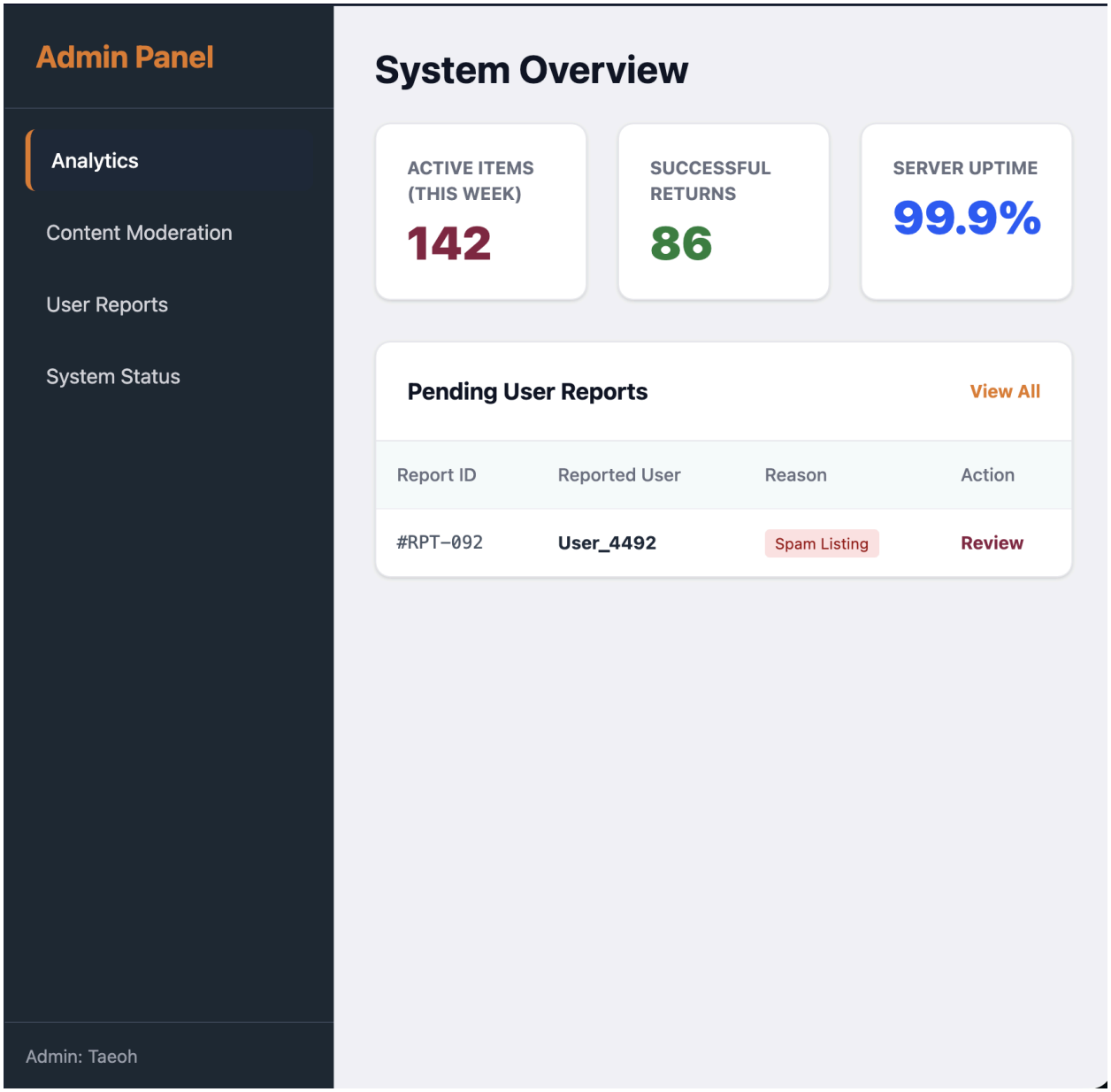
Update Status

Dorm Keys on VT Lanyard Resolved

Reported Lost on Oct 5 • Claimed & Returned

Archived


JD
Jane Doe
Re: AirPods Pro (Gen 2)
Report User
Today, 2:14 PM
JD
Hi! I think I found your AirPods. Could you tell me what the case looks like to confirm?
Yes! It has a small scratch on the bottom near the charging port, and a tiny dent on the right corner.
Type a message...
Send



2. **Algorithm Design expressed with pseudo-code:**

Algorithm 1: ReportLostItem
Input: formData (title, description, dateLost, locationLost, category, photos[])
Output: Confirmation message or error

```
1:  BEGIN
2:      /* Validate required fields */
```

```
3:    IF formData.title = NULL OR formData.description = NULL OR
4:      formData.dateLost = NULL OR formData.locationLost = NULL THEN
5:          DISPLAY "Missing required fields"
6:          RETURN
7:    ENDIF

8:    /* Validate photo count */
9:    IF length(photos[]) > 5 THEN
10:       DISPLAY "Maximum 5 photos allowed"
11:       RETURN
12:   ENDIF

13:   /* Create new lost item record */
14:   lostItem ← CreateLostItem(formData)

15:   IF lostItem = NULL THEN
16:       DISPLAY "Error creating lost item"
17:       RETURN
18:   ENDIF

19:   /* Update search index and notify system */
20:   CALL IndexItemForSearch(lostItem)
21:   CALL BroadcastUpdate("NewLostItem", lostItem.id)

22:   DISPLAY "Lost item successfully posted"
23: END
```

Algorithm 2: UpdateLostItemStatus
Input: itemId, newStatus
Output: Updated status or error

```
1:  BEGIN
2:    item ← QueryDB("LostItem", itemId)

3:    /* Verify ownership */
4:    IF item.ownerId ≠ currentUser.id THEN
5:       DISPLAY "Unauthorized action"
6:       RETURN
7:    ENDIF

8:    /* Validate status */
9:    IF newStatus NOT IN {Active, Found} THEN
10:       DISPLAY "Invalid status"
11:       RETURN
12:   ENDIF

13:   /* Update status */
14:   item.status ← newStatus
```

```
    SAVE item

    /* Notify claimants and sync system-wide */
    CALL NotifyPendingClaimants(itemId, newStatus)
    CALL BroadcastUpdate("LostItemStatusChanged", itemId)

    DISPLAY "Status updated successfully"
END
```

Algorithm 3: ViewClaimRequests
Input: itemId
Output: List of claim requests or message

```
BEGIN
    item ← QueryDB("LostItem", itemId)

    /* Verify ownership */
    IF item.ownerId ≠ currentUser.id THEN
        DISPLAY "Unauthorized access"
        RETURN
    ENDIF

    /* Retrieve claim requests */
    claims ← QueryDB("ClaimRequest WHERE itemId = itemId ORDER BY timestamp")

    IF claims is EMPTY THEN
        DISPLAY "No claims yet"
        RETURN
    ENDIF

    /* Display each claim */
    FOR each claim IN claims DO
        DISPLAY claim.claimantInfo, claim.timestamp, claim.status
    END FOR
END
```

Algorithm 4: AssignCategoryToLostItem
Input: itemId, category
Output: Updated item or error

```
BEGIN
    item ← QueryDB("LostItem", itemId)

    /* Verify ownership */
    IF item.ownerId ≠ currentUser.id THEN
        DISPLAY "Unauthorized action"
        RETURN
```

```
7:     ENDIF

8:     validCategories ← LoadCategoryList()

9:     /* Validate category */
10:    IF category NOT IN validCategories THEN
11:        IF category = "Other" THEN
12:            CALL PromptForCustomCategory()
13:        ELSE
14:            DISPLAY "Invalid category"
15:            RETURN
16:        ENDIF
17:    ENDIF

18:    /* Save category */
19:    item.category ← category
20:    SAVE item

21:    /* Update search index and sync system */
22:    CALL UpdateSearchIndex(item)
23:    CALL BroadcastUpdate("CategoryUpdated", itemId)

24:    DISPLAY "Category updated successfully"
25: END
```

Algorithm 5: ReportFoundItem

Input: formData (title, description, dateFound, locationFound, category, photos[])
Output: Confirmation message or error

```
1: BEGIN
2:     /* Validate required fields */
3:     IF formData.title = NULL OR formData.description = NULL OR
4:        formData.dateFound = NULL OR formData.locationFound = NULL THEN
5:        DISPLAY "Missing required fields"
6:        RETURN
7:     ENDIF

8:     /* Validate photo count */
9:     IF length(photos[]) > 5 THEN
10:        DISPLAY "Maximum 5 photos allowed"
11:        RETURN
12:    ENDIF

13:    /* Create new found item record */
14:    foundItem ← CreateFoundItem(formData, status = "Available")
15:    IF foundItem = NULL THEN
16:        DISPLAY "Error creating found item"
17:        RETURN
18:    ENDIF
```

```
19:   /* Update search and notify system */
20:   CALL IndexItemForSearch(foundItem)
21:   CALL BroadcastUpdate("NewFoundItem", foundItem.id)

22:   DISPLAY "Found item successfully posted"
23: END
```

Algorithm 6: UpdateFoundItemStatus

Input: itemId, newStatus
Output: Updated status or error

```
1: BEGIN
2:    item ← QueryDB("FoundItem", itemId)

3:    /* Verify ownership */
4:    IF item.finderId ≠ currentUser.id THEN
5:       DISPLAY "Unauthorized action"
6:       RETURN
7:    ENDIF

8:    /* Validate status */
9:    IF newStatus NOT IN {Available, Claimed} THEN
10:      DISPLAY "Invalid status"
11:      RETURN
12:   ENDIF

13:   /* Check proof requirement */
14:   IF newStatus = "Claimed" AND ProofVerificationRequired(itemId) = TRUE THEN
15:      DISPLAY "Proof verification required before claiming item"
16:      RETURN
17:   ENDIF

18:   /* Update status */
19:   item.status ← newStatus
20:   SAVE item

21:   CALL UpdateSearchIndex(item)
22:   CALL BroadcastUpdate("FoundItemStatusChanged", itemId)

23:   DISPLAY "Status updated successfully"
24: END
```

Algorithm 7: AcceptOrDeclineClaimRequest

Input: claimId, decision
Output: Updated claim status or error

```
1: BEGIN
2:    claim ← QueryDB("ClaimRequest", claimId)
3:    item ← QueryDB("FoundItem", claim.itemId)
```

```
4:    /* Verify ownership */
5:    IF item.finderId ≠ currentUser.id THEN
6:       DISPLAY "Unauthorized action"
7:       RETURN
8:    ENDIF

9:    /* Validate decision */
10:   IF decision NOT IN {Accept, Decline} THEN
11:      DISPLAY "Invalid decision"
12:      RETURN
13:   ENDIF

14:   /* Handle acceptance */
15:   IF decision = "Accept" THEN
16:      claim.status ← "PendingVerification"
17:      SAVE claim
18:      DISPLAY "Claim accepted. Proof verification required."
19:      RETURN
20:   ENDIF

21:   /* Handle decline */
22:   claim.status ← "Declined"
23:   SAVE claim
24:   CALL NotifyUser(claim.claimantId, "Your claim was declined")

25:   DISPLAY "Claim declined successfully"
26: END
```

Algorithm 8: VerifyProofOfOwnership
Input: claimId, verificationDecision
Output: Verified claim or error

```
1: BEGIN
2:    claim ← QueryDB("ClaimRequest", claimId)
3:    item ← QueryDB("FoundItem", claim.itemId)
4:    proof ← QueryDB("ProofOfOwnership", claimId)

5:    /* Verify ownership */
6:    IF item.finderId ≠ currentUser.id THEN
7:       DISPLAY "Unauthorized action"
8:       RETURN
9:    ENDIF

10:   /* Validate proof */
11:   IF proof = NULL THEN
12:      DISPLAY "No proof submitted"
13:      RETURN
14:   ENDIF
```

```
15:   /* Check item status */
16:   IF item.status ≠ "Available" THEN
17:      DISPLAY "Item already claimed"
18:      RETURN
19:   ENDIF

20:   /* Process verification */
21:   IF verificationDecision = "Verified" THEN
22:      claim.status ← "AcceptedVerified"
23:      item.status ← "Claimed"
24:      SAVE claim
25:      SAVE item

26:      CALL CloseOtherPendingClaims(item.id)
27:      CALL NotifyUser(claim.claimantId, "Your proof was verified")

28:      DISPLAY "Proof verified and item marked claimed"
29:   ELSE
30:      claim.status ← "Declined"
31:      SAVE claim

32:      CALL NotifyUser(claim.claimantId, "Proof not verified")

33:      DISPLAY "Proof not verified"
34:   ENDIF
35: END
```

Algorithm 9: AdvancedSearchAndFiltering
Input: query, filters (category, itemType, location, dateRange, status)
Output: Matching item list or message

```
1: BEGIN
2:    /* Validate filters */
3:    IF ConflictingFilters(filters) = TRUE THEN
4:       DISPLAY "Conflicting filters selected"
5:       RETURN
6:    ENDIF

7:    /* Run search */
8:    results ← SearchItems(query, filters)

9:    IF results is EMPTY THEN
10:      DISPLAY "No matches found"
11:      RETURN
12:   ENDIF

13:   /* Display results */
14:   results ← SortByRelevance(results)
```

```
15:   FOR each item IN results DO
16:       DISPLAY item.title, item.category, item.location, item.status
17:   END FOR
18: END
```

Algorithm 10: ItemMatching Recommendations
Input: itemId
Output: Ranked list of suggested matches or message

```
1: BEGIN
2:    item ← QueryDB("Item", itemId)

3:    /* Validate item data */
4:    IF item.category = NULL OR item.location = NULL OR item.dateReported = NULL THEN
5:       DISPLAY "Add more details to improve matching"
6:       RETURN
7:    ENDIF

8:    candidates ← QueryOppositeTypeItems(item)
9:    recommendations ← EMPTY_LIST

10:   FOR each candidate IN candidates DO
11:       score ← ComputeMatchScore(item, candidate)

12:       IF score ≥ MATCH_THRESHOLD THEN
13:          ADD (candidate, score, ExplainMatch(item, candidate)) TO recommendations
14:       ENDIF
15:   END FOR

16:   IF recommendations is EMPTY THEN
17:       DISPLAY "No recommendations yet"
18:       RETURN
19:   ENDIF

20:   recommendations ← SortDescendingByScore(recommendations)
21:   DISPLAY recommendations
22: END
```

Algorithm 11: SendMatchNotifications
Input: newItemId
Output: Notifications sent or logged

```
1: BEGIN
2:    newItem ← QueryDB("Item", newItemId)
3:    matches ← FindPotentialMatches(newItem)

4:    IF matches is EMPTY THEN
5:       RETURN
```

```
6:    ENDIF

7:    FOR each match IN matches DO
8:        user ← GetRelevantUser(match)

9:        IF user.notificationsEnabled = FALSE THEN
10:           CONTINUE
11:       ENDIF

12:       IF TooManyRecentMatches(user.id) = TRUE THEN
13:           CALL QueueBatchedNotification(user.id, match.id)
14:       ELSE
15:           CALL SendNotification(user.id, "New possible match found")
16:       ENDIF

17:       CALL LogNotification(user.id, match.id)
18:   END FOR
19: END
```

Algorithm 12: SendInAppMessage
Input: itemId, recipientId, messageText
Output: Sent message or error

```
1: BEGIN
2:    IF Trim(messageText) = "" THEN
3:        DISPLAY "Message cannot be empty"
4:        RETURN
5:    ENDIF

6:    IF MessagingDisabled(recipientId) = TRUE THEN
7:        DISPLAY "Messaging unavailable"
8:        RETURN
9:    ENDIF

10:   conversation ← FindConversation(itemId, currentUser.id, recipientId)

11:   IF conversation = NULL THEN
12:       conversation ← CreateConversation(itemId, currentUser.id, recipientId)
13:   ENDIF

14:   message ← CreateMessage(conversation.id, currentUser.id, messageText)

15:   IF message = NULL THEN
16:       DISPLAY "Message not sent. Try again."
17:       RETURN
18:   ENDIF

19:   DISPLAY "Message sent"
20: END
```

Algorithm 13: AccessAdminDashboardAndAnalytics

Input: adminId
Output: Dashboard data or error

```
1: BEGIN
2:    admin ← QueryDB("User", adminId)

3:    /* Verify admin privileges */
4:    IF admin.role ≠ "Admin" THEN
5:       DISPLAY "Access denied"
6:       CALL LogSecurityEvent(adminId, "Unauthorized dashboard access")
7:       RETURN
8:    ENDIF

9:    /* Retrieve analytics */
10:   activeItems ← CountActiveItems()
11:   successfulReturns ← CountSuccessfulReturns()
12:   userGrowth ← GetUserGrowthMetrics()
13:   uptime ← GetSystemUptime()

14:   /* Display dashboard */
15:   DISPLAY activeItems, successfulReturns, userGrowth, uptime
16: END
```

Algorithm 14: ModerateItemContent

Input: adminId, itemId, action
Output: Updated item moderation status or error

```
1: BEGIN
2:    admin ← QueryDB("User", adminId)
3:    item ← QueryDB("Item", itemId)

4:    /* Verify admin privileges */
5:    IF admin.role ≠ "Admin" THEN
6:       DISPLAY "Access denied"
7:       RETURN
8:    ENDIF

9:    /* Validate action */
10:   IF action NOT IN {DeletePost, HidePost} THEN
11:      DISPLAY "Invalid moderation action"
12:      RETURN
13:   ENDIF

14:   /* Apply moderation action */
15:   item.status ← "Removed"
```

```
16:    SAVE item

17:    CALL LogAdminAction(adminId, itemId, action)

18:    DISPLAY "Post moderated successfully"
19: END
```

Algorithm 15: ManageUserBehaviorReports

Input: adminId, reportId, decision
Output: Updated report resolution or error

```
1: BEGIN
2:     admin ← QueryDB("User", adminId)
3:     report ← QueryDB("UserReport", reportId)

4:     /* Verify admin privileges */
5:     IF admin.role ≠ "Admin" THEN
6:         DISPLAY "Access denied"
7:         RETURN
8:     ENDIF

9:     /* Validate decision */
10:    IF decision NOT IN {Dismiss, WarnUser, SuspendAccount} THEN
11:        DISPLAY "Invalid decision"
12:        RETURN
13:    ENDIF

14:    /* Apply decision */
15:    IF decision = "Dismiss" THEN
16:        report.status ← "Dismissed"

17:    ELSE IF decision = "WarnUser" THEN
18:        CALL SendWarning(report.targetUserId)
19:        report.status ← "Warned"

20:    ELSE
21:        CALL SuspendUser(report.targetUserId)
22:        report.status ← "Suspended"
23:    ENDIF

24:    SAVE report

25:    CALL LogAdminAction(adminId, reportId, decision)
26:    CALL NotifyUser(report.reportingUserId, "Report resolved")

27:    DISPLAY "Report processed successfully"
28: END
```

Algorithm 16: MonitorSystemPerformance

Input: adminId
Output: Performance metrics or error

```
1: BEGIN
2:    admin ← QueryDB("User", adminId)

3:    /* Verify admin privileges */
4:    IF admin.role ≠ "Admin" THEN
5:       DISPLAY "Access denied"
6:       RETURN
7:    ENDIF

8:    /* Retrieve system metrics */
9:    uptime ← GetSystemUptime()
10:   dbResponse ← GetDatabaseResponseTime()
11:   syncStatus ← GetSynchronizationStatus()
12:   socketCount ← GetActiveWebSocketCount()

13:   DISPLAY uptime, dbResponse, syncStatus, socketCount

14:   /* Detect performance issues */
15:   IF dbResponse > RESPONSE_THRESHOLD OR syncStatus = "Error" THEN
16:      DISPLAY "Performance issue detected"
17:      CALL RunDiagnostics()
18:   ENDIF
19: END
```

Algorithm 17: RegisterUser

Input: fullName, email, password
Output: Account created or error

```
1: BEGIN
2:    /* Validate required fields */
3:    IF fullName = NULL OR email = NULL OR password = NULL THEN
4:       DISPLAY "Missing required fields"
5:       RETURN
6:    ENDIF

7:    /* Validate university email */
8:    IF NOT IsValidUniversityEmail(email) THEN
9:       DISPLAY "Invalid university email format"
10:      RETURN
11:   ENDIF

12:   /* Check duplicate account */
13:   IF EmailExists(email) = TRUE THEN
14:      DISPLAY "Email already registered"
```

```
15:       RETURN
16:    ENDIF

17:    /* Create user */
18:    user ← CreateUser(fullName, email, HashPassword(password))

19:    CALL SendVerificationLink(user.email)

20:    DISPLAY "Verification email sent"
21: END
```

Algorithm 18: LoginUser

Input: email, password
Output: Authenticated session or error

```
1: BEGIN
2:     user ← QueryUserByEmail(email)

3:     IF user = NULL THEN
4:        DISPLAY "Invalid login credentials"
5:        RETURN
6:     ENDIF

7:     /* Check lock status */
8:     IF user.lockedUntil > CURRENT_TIME THEN
9:        DISPLAY "Account temporarily locked"
10:       RETURN
11:    ENDIF

12:    /* Verify password */
13:    IF VerifyPassword(password, user.passwordHash) = FALSE THEN
14:       user.failedAttempts ← user.failedAttempts + 1

15:       IF user.failedAttempts ≥ 5 THEN
16:          user.lockedUntil ← CURRENT_TIME + 15 minutes
17:       ENDIF

18:       SAVE user
19:       DISPLAY "Incorrect password"
20:       RETURN
21:    ENDIF

22:    /* Successful login */
23:    user.failedAttempts ← 0
24:    sessionToken ← GenerateSecureSessionToken(user.id)

25:    SAVE user
```

```
26:    DISPLAY "Login successful"
27: END
```

Algorithm 19: UpdateUserProfile

Input: profileData (phone, secondaryEmail, profilePicture, contactPreference)
Output: Updated profile or error

```
1: BEGIN
2:     user ← QueryDB("User", currentUser.id)

3:     /* Validate phone */
4:     IF profileData.phone ≠ NULL AND NOT IsValidPhone(profileData.phone) THEN
5:         DISPLAY "Invalid phone number format"
6:         RETURN
7:     ENDIF

8:     /* Update fields */
9:     user.phone ← profileData.phone
10:    user.secondaryEmail ← profileData.secondaryEmail
11:    user.profilePicture ← profileData.profilePicture
12:    user.contactPreference ← profileData.contactPreference

13:    SAVE user

14:    DISPLAY "Profile updated successfully"
15: END
```

Algorithm 20: ManageMyItemsDashboard

Input: userId
Output: User item list or message

```
1: BEGIN
2:     items ← QueryDB("Item WHERE ownerId = userId OR finderId = userId")

3:     IF items is EMPTY THEN
4:         DISPLAY "No posts yet"
5:         RETURN
6:     ENDIF

7:     /* Display items */
8:     FOR each item IN items DO
9:         DISPLAY item.title, item.type, item.status, item.dateReported
10:    END FOR

11:    /* Refresh if needed */
12:    IF HasPendingUpdates(userId) = TRUE THEN
13:        CALL RefreshDashboard(userId)
14:    ENDIF
```

15: END